

تطوير تطبيق مراسلة آمن

تقرير مشروع (تخرج ١) – قدم استكمالاً لمتطلبات الحصول على درجة البكالوريوس

في هندسة المعلوماتية – هندسة امن النظم و الشبكات الحاسوبية

إعداد :

محمد مالك غنام

إشراف :

الدكتورة كريستين زينية

٢٠٢٤ - ٢٠٢٥

أشهد بأن إعداد هذا المشروع الموسوم

والمعد من قبل الطالب:

قد تم تحت إشرافي في قسم هندسة أمن النظم والشبكات الحاسوبية – كلية هندسة الحاسوب

والمعلوماتية والاتصالات وهو جزء من متطلبات نيل شهادة الإجازة في هندسة أمن النظم

والشبكات الحاسوبية.

التوقيع

الاسم

المرتبة العلمية

التاريخ

يتناول هذا البحث تطوير نظام مراسلة فورية آمن يعتمد على بروتوكول تشفير هجين يجمع بين خوارزميات تقليدية ومقاومة للهجمات الكمومية. جاءت أهمية البحث نتيجة لتزايد المخاطر الأمنية المصاحبة لظهور الحوسبة الكمومية، وعجز الأنظمة التقليدية عن توفير الحماية الكافية للرسائل الرقمية ضد الهجمات المستقبلية. يهدف البحث إلى بناء نظام يمكنه تأمين تبادل الرسائل النصية بين المستخدمين بشكل يضمن سرية المعلومات وعدم قدرة أي طرف ثالث على الاطلاع عليها، حتى في حال اختراق الخادم أو توفر موارد حسابية متقدمة لدى المهاجم.

تعتمد المنهجية على استخدام بروتوكول تشفير يجمع بين خوارزمية X3DH لتبادل المفاتيح الآمن بخوارزمية Kyber (مقاومة للهجمات الكمومية) وخوارزمية Double Ratchet لتشفير كل رسالة على حدة باستخدام مفتاح متغير باستمرار. تم تطبيق النظام باستخدام لغة بايثون وبيئة Flask، مع تخزين الرسائل بحالتها المشفرة فقط على الخادم، بحيث لا يمكن لأي جهة غير أطراف المحادثة فك تشفيرها. اعتمدت الدراسة على اختبارات عملية لتحليل مقاومة النظام لهجمات التنصت، اعتراض الرسائل، وهجمات كسر المفاتيح الكمومية، إضافة إلى دراسة متطلبات الأداء ومرونة التكامل.

أظهرت النتائج أن النظام يوفر سرية عالية للرسائل النصية، ويمنع جميع أشكال التنصت والوصول غير المصرح به حتى في أسوأ السيناريوهات الأمنية. كما أكدت النتائج أن كل رسالة يتم تشفيرها بمفتاح مختلف، بما يمنع فك تشفير أي رسائل أخرى حتى في حال تسريب جزء من مفاتيح الجلسة، مع المحافظة على الأداء والسرعة ضمن معايير مقبولة لتطبيقات المراسلة.

Abstract

This research addresses the development of a secure instant messaging system based on a hybrid cryptographic protocol that combines both classical and post-quantum algorithms. The importance of this study arises from the growing security threats associated with the advent of quantum computing and the inability of traditional systems to provide sufficient protection for digital messages against future attacks. The aim of this project is to build a system capable of ensuring the confidentiality of textual message exchanges between users, guaranteeing that no third party can access the information—even if the server is compromised or attackers possess advanced computational resources.

The methodology relies on a cryptographic protocol that integrates the X3DH key exchange (which employs the quantum-resistant Kyber algorithm) with the Double Ratchet algorithm, thus encrypting each message individually with continuously evolving session keys. The system is implemented using Python and Flask, with all messages stored in their encrypted state on the server, ensuring that only the communicating parties can decrypt the content. The study includes practical tests to analyze the system's resistance to eavesdropping, message interception, and quantum key-recovery attacks, in addition to assessing performance requirements and integration flexibility.

The results demonstrate that the system achieves a high level of confidentiality for textual messages and prevents all forms of eavesdropping and unauthorized access, even under severe security scenarios. Furthermore, each message is encrypted with a unique key, ensuring that the compromise of any single session key does not affect the security of other messages. The system maintains an acceptable level of performance and speed suitable for practical messaging applications.

الاختصار	التعريف
E2EE	End-to-End Encryption
PQX3DH	Post-Quantum Extended Triple Diffie-Hellman
Kyber	Kyber Post-Quantum KEM
ECC	Elliptic Curve Cryptography
AES	Advanced Encryption Standard
XChaCha20	XChaCha20 stream cipher
Double Ratchet	Double Ratchet key update
OPK	One-time PreKey
SPK	Signed PreKey
IK	Identity Key
KEM	Key Encapsulation Mechanism

الفصل الأول

مقدمة

١. المقدمة

مع التسارع الكبير في تطور تكنولوجيا المعلومات وانتشار تطبيقات الاتصال الرقمي، تبرز الحاجة الملحة إلى حماية خصوصية المستخدمين وسرية بياناتهم، خصوصاً مع ازدياد الهجمات السيبرانية والتنصت التي تستهدف أنظمة الاتصالات والمراسلات. أصبحت تطبيقات التراسل الفوري جزءاً لا يتجزأ من الحياة اليومية للأفراد، والمؤسسات، والحكومات، حيث تُستخدم لتبادل بيانات حساسة وشخصية، تشمل كلمات المرور، والمحادثات الخاصة، والمستندات، والمعاملات الرقمية، وغير ذلك من البيانات ذات الطابع السري.

في هذا السياق، بات من الضروري التفكير في تطوير حلول أمنية متقدمة، تشمل الحماية من جميع أنواع التهديدات الكلاسيكية وحتى التهديدات المستقبلية المتمثلة في مجال الحوسبة الكمومية التي لم تنتشر بعد، ولكن انتشارها يهدد الأسس الرياضية التي تعتمد عليها خوارزميات التشفير الكلاسيكية مثل RSA و ECDH. هذه الخوارزميات معرضة وغيرها للخطر بسبب قدرة الحواسيب الكمومية المستقبلية على كسرها خلال وقت زمني قصير باستخدام خوارزميات مثل Shor و Grover، مما يعني أن أي نظام أمني يعتمد عليها حصرًا قد يصبح غير آمن في المستقبل القريب.

بناءً عليه، جاءت فكرة المشروع وهي تطوير نظام تراسل آمن يجمع بين خوارزميات التشفير الكلاسيكية مثل Diffie-Hellman (ECDH) وخوارزميات ما بعد الكم مثل Kyber، وتغليف هذا النظام ضمن بروتوكول تشفير قوي يضمن سرية الرسائل، وسلامتها، ومقاومتها لهجمات الأطراف الوسيطة (MITM)، وحتى القدرة على حماية الرسائل السابقة والمستقبلية عبر مفهوم السرية الأمامية (Forward Secrecy) والسرية المستقبلية (Future Secrecy).

٢. أهداف المشروع

- تطوير بروتوكول تشفير آمن يجمع بين خوارزميات كلاسيكية وما بعد كمومية، يضمن سرية وأمان تبادل الرسائل النصية.
- تصميم تطبيق مراسلة فوري يعمل محليًا بواجهة رسومية، يطبق بروتوكول التشفير بشكل عملي ويوضح كيفية تبادل الرسائل بأمان تام.
- تحقيق خاصيتي السرية الأمامية والمستقبلية عبر استخدام بروتوكول Double Ratchet الذي يتميز بتحديث المفاتيح بعد كل رسالة.
- التصدي لهجمات التنصت والهجمات الوسيطة من خلال تبادل المفاتيح بطريقة موثوقة باستخدام توقيعات رقمية آمنة.

الفصل الثاني

الإطار النظري

يستعرض هذا الفصل الجوانب النظرية الأساسية المرتبطة بتطوير أنظمة المحادثة الآمنة، مع التركيز على المفاهيم المتعلقة بتأمين الاتصالات الرقمية مثل التشفير المتناظر و غير المتناظر و التشفير من طرف إلى طرف وآليات تبادل المفاتيح. كما يتناول البروتوكولات التي تعتمد تطبيقات المحادثة الحديثة، مثل بروتوكول Signal وخوارزمية Double Ratchet، إضافة إلى التشفير المقاوم للهجمات الكمومية مثل Kyber. يهدف هذا الفصل إلى توضيح الأسس النظرية والتقنيات التي تم الاعتماد عليها في بناء النظام، وبيان أهمية كل خوارزمية أو تقنية مستخدمة في تعزيز أمان وخصوصية البيانات المتبادلة بين المستخدمين.

٢. التشفير (Cryptography)

يُعرّف التشفير بأنه العلم الذي يُعنى بتطوير واستخدام تقنيات رياضية تهدف إلى حماية المعلومات من الوصول أو التلاعب غير المصرّح به، وذلك من خلال تحويل البيانات الأصلية (النص الصريح) إلى صيغة غير مفهومة (النص المشفر)، بحيث لا يمكن استرجاعها إلا باستخدام مفتاح خاص [١].

في هذا المشروع يعتبر "التشفير" هو اساس لبناء تطبيق لتبادل الرسائل بشكل امن.

٣. التشفير المتناظر (Symmetric Encryption)

هو أسلوب تشفير يعتمد على استخدام نفس المفتاح في عمليتي التشفير وفك التشفير. من أشهر خوارزميات التشفير المتناظر: AES و ChaCha20. يتميز بسرعته وكفاءته، لكنه يتطلب وسيلة آمنة لتبادل المفتاح السري بين الطرفين [١].

في هذا المشروع استخدم التشفير المتناظر في تشفير الرسائل بين الطرفين .

٤. التشفير غير المتناظر (Asymmetric Encryption)

يعتمد هذا النوع على زوج من المفاتيح: مفتاح عام للتشفير، ومفتاح خاص لفك التشفير. أشهر خوارزمياته: RSA، و ECC. يسهّل التشفير غير المتناظر عملية تبادل المفاتيح عبر قنوات غير آمنة، ويستخدم غالبًا لتبادل مفاتيح الجلسة في الأنظمة الهجينة [١][٤].

في هذا المشروع استخدم التشفير غير المتناظر في عملية تبادل المفاتيح للاتفاق على مفتاح سر مشترك .

٥. التشفير الهجين (Hybrid Encryption)

يُقصد بالتشفير الهجين الجمع بين أسلوبَي التشفير المتناظر وغير المتناظر للاستفادة من مزايا كل منهما. عادةً، يُستخدم التشفير غير المتناظر (مثل RSA أو ECC) لتبادل مفتاح جلسة سري بين الطرفين عبر قناة غير آمنة، ثم يُستخدم هذا المفتاح لاحقًا في تشفير وفك تشفير جميع الرسائل باستخدام خوارزمية متناظرة سريعة (مثل AES أو ChaCha20).

هذه الطريقة تضمن أمان تبادل المفاتيح وسرعة تشفير البيانات في نفس الوقت، وهي مطبقة في معظم بروتوكولات التشفير الحديثة ك X3DH [2][1].

٦. سرية أمامية (Forward Secrecy)

هي خاصية أمنية تضمن أنه في حال تم تسريب أحد مفاتيح الجلسة، فإن الرسائل السابقة تظل آمنة وغير قابلة للفك، لأن كل رسالة أو جلسة تستخدم مفتاحًا مؤقتًا مختلفًا [٧].

مفهوم ال (forward secrecy) مطبق عن طريق بروتوكول double ratchet المستخدم .

٧. السرية ما بعد الاختراق (Post-Compromise Security)

هي خاصية أمنية متقدمة في بروتوكولات التشفير تضمن أنه حتى إذا تمكّن المهاجم من اختراق جهاز أحد الأطراف واكتشاف المفاتيح السرية في لحظة معينة، سيبقى بإمكان الأطراف استعادة سرية الاتصال تلقائيًا بعد فترة قصيرة من تبادل الرسائل، دون الحاجة إلى إعادة إنشاء الجلسة يدويًا أو تغيير جميع المفاتيح.

تُعرف هذه الخاصية أيضًا باسم "السرية المستقبلية" (Future Secrecy)، وتتحقق من خلال آليات تحديث مستمر للمفاتيح كما هو الحال في خوارزمية Double Ratchet المستخدمة في بروتوكول Signal، حيث يتم تجديد المفتاح بعد كل رسالة تقريبًا. بذلك، تقتصر فترة الخطر على فترة سيطرة المهاجم الفعلية فقط، وتعود سرية الاتصال فورًا بعد أول رسالة غير معترضة بعد إزالة التهديد [٧].

مفهوم السرية ما بعد الاختراق مطبق عن طريق بروتوكول double ratchet المستخدم .

٨. المصادقة (Authentication)

المصادقة هي عملية تهدف إلى التأكد من هوية الأطراف المشاركة في الاتصال أو تبادل البيانات، بحيث تضمن أن كل طرف في عملية الاتصال هو فعلاً من يدّعي أنه كذلك، وليس طرفًا مزيفًا أو متطفلاً.

تعتمد المصادقة غالبًا على استخدام مفاتيح سرية، أو كلمات مرور، أو رموز تحقق، أو التوقيع الرقمي في الأنظمة الحديثة. في بروتوكولات التشفير مثل X3DH و Signal، تعتبر المصادقة عاملاً رئيسيًا لمنع هجمات "رجل في الوسط" (MitM) وضمان أمان تبادل المفاتيح وتشفير الرسائل [١][٢].

في هذا المشروع تم تحقيق خاصية ال (Authentication) أولا في عملية تبادل المفاتيح عن طريق X3DH و اثناء عملية تشفير الرسائل في استخدام خوارزمية Xchacha20 – poly1305 .

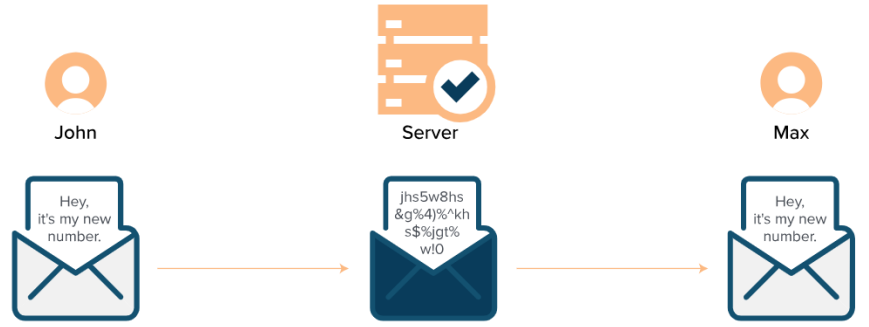
٩. التشفير من طرف إلى طرف (End-to-End Encryption)

يعد التشفير من طرف إلى طرف (End-to-End Encryption - E2EE) أحد الأسس الجوهرية لحماية خصوصية الاتصالات الرقمية، خصوصًا في تطبيقات المراسلة الفورية. يُعرّف E2EE بأنه نظام تشفير تُشفّر فيه الرسائل بحيث لا يستطيع قراءة محتواها سوى الطرفان المشاركان في الاتصال، ولا يمكن لأي وسيط — بما في ذلك مزود الخدمة — الاطلاع على الرسائل أثناء نقلها عبر الشبكة [1].

يعتمد E2EE على أن الرسالة تُشفّر عند المرسل ولا تُفك إلا عند المستقبل كما هو موضح في الشكل ٢،١ ، ويتم ذلك من خلال توليد وتبادل مفاتيح سرية باستخدام بروتوكولات متقدمة مثل X3DH و Double Ratchet [2] . هذا يضمن سرية الرسائل حتى في حال اعتراضها من طرف ثالث، ويجعل من الصعب جدًا على أي جهة الوصول للمحتوى بدون امتلاك المفاتيح الصحيحة.

لذلك، تعتمد تطبيقات المراسلة الحديثة مثل WhatsApp و Signal على التشفير من طرف إلى طرف كعنصر أساسي في بنيتها الأمنية، لضمان عدم تسرب المعلومات الحساسة وحماية المستخدمين من التنصت [2].

في هذا المشروع تم تطبيق خاصية التشفير من طرف الى طرف .



٢،١ . توضيح التشفير من طرف الى طرف

١٠. تبادل المفاتيح بطريقة ديفي-هيلمان (Diffie-Hellman Key Exchange)

بروتوكول ديفي-هيلمان هو واحد من أوائل بروتوكولات تبادل المفاتيح في علم التشفير الحديث. يسمح لطرفين أو أكثر بتوليد مفتاح سري مشترك عبر قناة عامة غير آمنة، بحيث يمكن استخدام هذا المفتاح لاحقًا لتشفير الرسائل بينهما باستخدام خوارزميات التشفير المتناظر.

يعتمد البروتوكول على صعوبة حل مسألة اللوغاريتم المنفصل في مجموعة رياضية كبيرة (مثل الأعداد الأولية الكبيرة أو المنحنيات البيضاوية)، ويشكّل الأساس النظري للعديد من بروتوكولات التشفير الحديثة مثل X3DH و Double Ratchet [1].

استخدمت عملية (Diffie-Hellman Key Exchange) ك مرحلة من مراحل بروتوكول تبادل المفاتيح X3DH المستخدم .

١١. بروتوكول X3DH (Extended Triple Diffie-Hellman)

يُعد بروتوكول X3DH من أهم بروتوكولات تبادل المفاتيح في تطبيقات التشفير من طرف إلى طرف، حيث يُستخدم لإنشاء سر مشترك بين طرفين حتى لو لم يكونا متصلين بالإنترنت في نفس اللحظة. تعتمد آلية عمل X3DH على أربع مفاتيح أساسية:

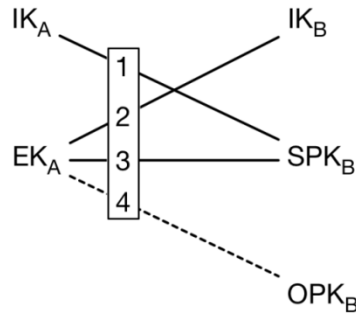
- مفتاح الهوية (Identity Key)
- مفتاح التوثيق الموقع (Signed Prekey)
- مفاتيح أحادية الاستخدام (One-Time Prekeys)
- مفتاح التشفير اللحظي (Ephemeral Key)

يتم تسجيل مفاتيح المستخدم مسبقاً على الخادم. عند بدء المحادثة، يقوم الطرف البادئ بجمع الحزم المفتاحية للطرف الآخر، وينفذ عدة عمليات Diffie-Hellman بينها كما هو موضح في الشكل ٢,٢ ، ثم يجمع النتائج لتوليد سر الجلسة باستخدام مشتقات المفاتيح [3][2] (HKDF).

تتميز X3DH بالقدرة على توفير سرية أمامية (Forward Secrecy) حتى في السيناريوهات التي يتغير فيها المستخدمون أو الأجهزة، كما أن استخدام مفاتيح أحادية الاستخدام يزيد الأمان ضد هجمات إعادة الإرسال.

يعد X3DH العمود الفقري لأي نظام يريد تأمين المصافحة الأولية بطريقة عملية ومرنة، لذلك اعتمدته تطبيقات عالمية مثل Signal و WhatsApp [3].

في هذا المشروع تم الاعتماد على بروتوكول X3DH في عملية تبادل المفاتيح مدموجاً بخوارزمية kyber .



٢,٢. مخطط عمليات DH في X3DH [٣]

١٢. خوارزمية Double Ratchet

تُعد خوارزمية Double Ratchet من الابتكارات الجوهرية في مجال تأمين الاتصالات الرقمية، حيث تمثل الطبقة المسؤولة عن تحديث المفاتيح بشكل مستمر بعد المصافحة الأولية، أي أثناء تبادل الرسائل بين الطرفين.

تعتمد Double Ratchet على مبدأ دمج سلسلتين من التحديثات:

سلسلة Diffie-Hellman، لتوليد أسرار جديدة عند كل تغيير في مفاتيح الطرفين.

سلسلة KDF (مشتقات المفاتيح)، لضمان وجود مفتاح فريد لكل رسالة فردية يتم إرسالها أو استقبالها [٢][٣].

تقوم الخوارزمية بتوليد مفاتيح جديدة بشكل تلقائي مع كل رسالة صادرة أو واردة، بحيث إذا تم اختراق أحد المفاتيح أو الأجهزة لاحقاً، لن يستطيع المهاجم الوصول للرسائل السابقة أو المستقبلية، وهو ما يعرف بخصائص السرية الأمامية وسرية ما بعد الاختراق [٣].

هذه الآلية تُكسب النظام قدرة عالية على مقاومة مختلف أنواع الهجمات، حتى لو تعرض أحد الأطراف للاختراق مؤقتاً، إذ تعود المحادثة لتصبح آمنة تلقائياً بعد التبادل الأول لأي رسالة جديدة.

تتكامل Double Ratchet مع بروتوكولات المصافحة (مثل X3DH أو PQX3DH) لتشكيل بنية أمان شاملة، ما يجعل تطبيقها أساساً في أنظمة التراسل الحديثة مثل Signal، كما تم اعتمادها بشكل مباشر في هذا المشروع لتحقيق أعلى معايير الأمان والاستمرارية في حماية الرسائل.

تعتبر خوارزمية Double Ratchet في هذا المشروع حجر الأساس في بناء بروتوكول التشفير

١٣. التشفير البيضاوي (Elliptic Curve Cryptography - ECC)

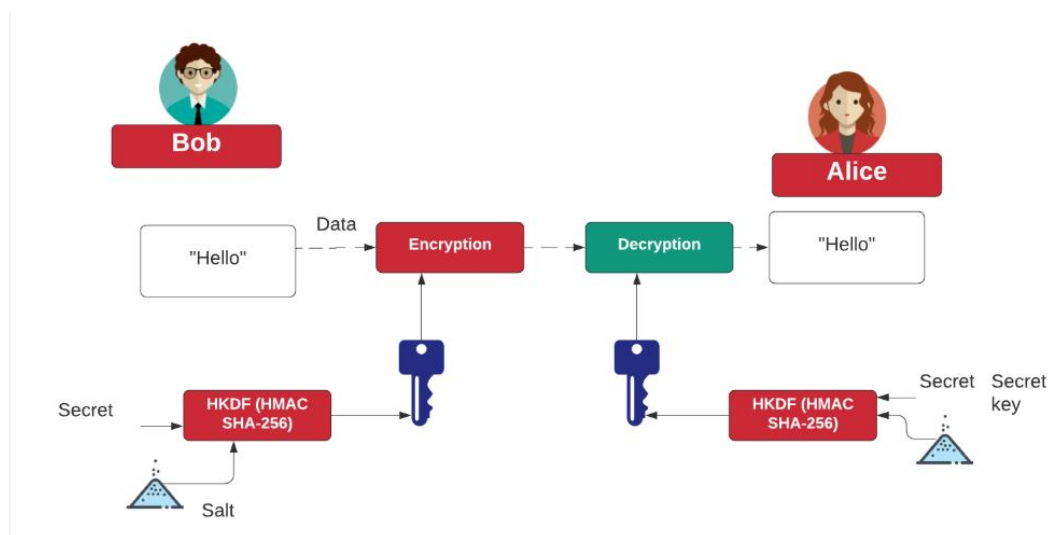
يعد التشفير باستخدام المنحنيات البيضاوية أو الإهليلجية (ECC) من أهم تقنيات التشفير الحديثة، ويتميز بقوة أمانه مقارنة بطول المفتاح، مما يجعله مثالياً لتطبيقات الأجهزة المحمولة وأنظمة الاتصالات الآمنة. تعتمد آلية ECC على صعوبة حل مسألة اللوغاريتم المنفصل على المنحنيات البيضاوية، وهي مشكلة رياضية لم تُحل حتى الآن بفاعلية من خلال الحواسيب التقليدية، وتُستخدم في العديد من بروتوكولات تبادل المفاتيح والتوقيع الرقمي [١][٢].

في هذا المشروع ، استخدم ECC في عملية DH أثناء تبادل المفاتيح في بروتوكول X3DH

١٤. مشتق المفاتيح (HKDF (HMAC-based Key Derivation Function)

تُستخدم دالة مشتق المفاتيح HKDF لتوليد مفاتيح سرية متعددة وأمنة انطلاقاً من مادة سرية رئيسية واحدة (Master Secret)، وذلك من خلال تمريرها على عدة مراحل من التلخيص والتجزئة عبر دالة HMAC. تُعتبر HKDF حجر الأساس في تصميم معظم بروتوكولات التشفير الحديثة، حيث تضمن أن كل مفتاح مشتق يكون مستقلاً وآمناً للاستخدام في عمليات تشفير منفصلة [٢][٣].

في هذا المشروع، استخدم مشتق المفاتيح HKDF في اشتقاق ناتج مفتاح دخل لبروتوكول double ratchet من خرج بروتوكول PQX3DH.



٢، ٣. مثال عن عمل دالة HKDF

١٥. XChaCha20-Poly1305

تُعتبر خوارزمية XChaCha20-Poly1305 تطويراً لخوارزمية ChaCha20-Poly1305، حيث تم تصميمها خصيصاً لمعالجة متطلبات الأمان الإضافية في بيئات تحتاج إلى أنوات (nonces) طويلة وتجنب احتمالية تكرار الأنوات بشكل فعال.

تستخدم XChaCha20 دالة ChaCha20 كأساس للتشفير المتناظر وتضيف إليها إمكانية استخدام نونس بطول ١٩٢ بت (٢٤ بايت)، مما يتيح إنشاء أنوات عشوائية وقوية حتى مع تشفير عدد هائل من الرسائل، وهو أمر بالغ الأهمية في الأنظمة التي تتطلب أماناً عالياً على مدى طويل [٤].

بالإضافة إلى ذلك، توفر XChaCha20-Poly1305 مصادقة للرسائل باستخدام Poly1305، بحيث تضمن سلامة البيانات وعدم التلاعب بها أثناء النقل. تُستخدم هذه الخوارزمية بشكل متزايد في التطبيقات الحديثة نظراً لسرعتها وسهولة تطبيقها البرمجي، وقدرتها على العمل بكفاءة عالية على الأجهزة محدودة الموارد [٥].

في هذا المشروع استخدمت خوارزمية XChaCha20-Poly1305 في عملية تشفير الرسائل بمفتاح فريد لكل رسالة من double ratchet

١٦. الحوسبة الكمومية (Quantum Computing)

الحوسبة الكمومية هي نموذج جديد للحوسبة يعتمد على مبادئ ميكانيكا الكم، حيث تستخدم الكيوبتات (qubits) بدلاً من البتات التقليدية لتمثيل ومعالجة المعلومات.

تتميز الحواسيب الكمومية بقدرتها على إجراء عمليات حسابية معقدة بسرعة هائلة مقارنة بالحواسيب الكلاسيكية، خاصة في مسائل مثل تحليل الأعداد الأولية وحل مشاكل اللوغاريتم المنفصل.

هذه القدرات تجعل الحوسبة الكمومية تهديدًا فعليًا لأنظمة التشفير التقليدية مثل RSA و ECC، ولذلك ظهرت الحاجة إلى تطوير خوارزميات مقاومة للكم مثل Kyber [4].

١٧. خوارزمية شور (Shor's Algorithm)

خوارزمية شور هي خوارزمية كمومية ابتكرها عالم الرياضيات بيتر شور في عام ١٩٩٤، وتعد من أكثر الخوارزميات الكمومية شهرة وتأثيرًا في عالم التشفير.

تهدف الخوارزمية إلى تحليل الأعداد الصحيحة الكبيرة إلى عواملها الأولية (Prime Factorization) بكفاءة عالية جدًا مقارنة بأي خوارزمية كلاسيكية.

في حين تستغرق الخوارزميات التقليدية سنوات لتحليل الأعداد الكبيرة (كما في خوارزمية RSA)، تستطيع خوارزمية شور إنجاز ذلك في وقت قصير جدًا إذا توفرت حواسيب كمومية قوية.

هذا التقدم يجعل جميع أنظمة التشفير القائمة على صعوبة تحليل الأعداد الأولية (مثل RSA و DSA) عرضة للخطر في المستقبل القريب، وهو ما دفع الباحثين لتطوير خوارزميات مقاومة للكم مثل Kyber وغيرها [٤].

١٨. التشفير ما بعد الكم (Post-Quantum Cryptography)

التشفير ما بعد الكم هو مجال في علم التشفير يهدف إلى تطوير خوارزميات وأنظمة تشفير يمكنها مقاومة الهجمات التي تنفذها الحواسيب الكمومية.

تركز هذه الخوارزميات على مشاكل رياضية يصعب حلها حتى باستخدام التقنيات الكمومية، مثل الشبكات الرياضية (lattices) والتواقيع الرقمية المعتمدة على الهاش.

أصبح هذا المجال ضروريًا لأن خوارزميات كمومية مثل شور (Shor's Algorithm) يمكن أن تكسر أنظمة التشفير التقليدية مثل RSA و ECC بسهولة، مما قد يهدد سرية البيانات حول العالم عند توفر حواسيب كمومية قوية.

ولهذا السبب بدأت مؤسسات دولية مثل NIST باعتماد خوارزميات تشفير ما بعد الكم مثل Kyber كمعايير جديدة للأمان المستقبلي [٤].

١٩. خوارزمية Kyber

خوارزمية Kyber هي واحدة من أحدث خوارزميات التشفير ما بعد الكم (Post-Quantum Cryptography) التي طوّرها فريق دولي من الباحثين، وتم اختيارها واعتمادها رسميًا من قبل معهد المعايير والتقنية الأمريكي (NIST) كمعيار جديد لتأمين تبادل المفاتيح في عصر الحوسبة الكمومية [٤].

تندرج Kyber ضمن عائلة خوارزميات "التشفير الشبكي" (Lattice-based Cryptography)، وتعتمد على مشكلة رياضية صعبة تُعرف باسم (Module Learning with Errors (Module-LWE)، والتي ثبت رياضياً صعوبة حلها حتى باستخدام الحواسيب الكمومية، ما يجعلها مقاومة لهجمات كمومية مستقبلية.

تُستخدم Kyber كآلية "تغليف مفتاح" (Key Encapsulation Mechanism - KEM)، أي أنها تتيح لطرفين الاتفاق على سر مشترك بأمان حتى في وجود قناة غير آمنة، وتستبدل بذلك خوارزميات مثل RSA أو Diffie-Hellman التي لم تعد آمنة أمام هجمات شور الكمومية.

في هذا المشروع تم استخدام خوارزمية kyber في عملية تبادل المفاتيح لضمان مقاومة حقيقية ضد الهجمات الكمومية

٢٠. مشاكل الشبكات الرياضية (Lattice Problems)

مشاكل الشبكات الرياضية (Lattice Problems) هي فئة من المسائل الرياضية التي تدور حول البنى الهندسية المعروفة باسم "الشبكات" (lattices)، وهي مجموعات لا نهائية من النقاط المنتظمة في الفضاء n -الأبعاد، يُمكن تمثيلها كمجموعات خطية صحيحة لمجموعة من المتجهات.

في علم التشفير، أصبحت مشاكل الشبكات ذات أهمية كبيرة لأنها تُعتبر صعبة الحل حتى بالنسبة لأقوى الحواسيب الكلاسيكية والكمومية، خاصة مشاكل مثل (Shortest Vector Problem (SVP و (Learning With Errors (LWE).

تُعد خوارزميات التشفير المعتمدة على مشاكل الشبكات (lattice-based cryptography) من الخيارات الأساسية لتصميم أنظمة مقاومة للكم، مثل خوارزمية Kyber، لأن كل المحاولات المعروفة حتى الآن لم تنجح في كسر هذه المشاكل بكفاءة عالية، حتى باستخدام الحوسبة الكمومية [١١].

الفصل الثالث

الدراسات المرجعية

يستعرض هذا القسم أبرز الدراسات والمشاريع السابقة التي تناولت موضوع التراسل الآمن وتطوير تطبيقات المحادثة المشفرة، بهدف الوقوف على الأساليب والتقنيات التي تم اعتمادها عملياً وأكاديمياً، بالإضافة إلى تحديد نقاط القوة والقصور في هذه الأعمال. يساعد تحليل هذه الدراسات في تبرير الحاجة إلى المشروع المقترح ويوضح أوجه التميز فيه.

٢. Secure Chat Application with an End-to-End Encryption

الملخص [٧]

تستعرض هذه الدراسة تجربة تصميم وبناء تطبيق ترسل فوري يركز على حماية الرسائل بين المستخدمين، باستخدام خوارزميتي RSA للتشفير غير المتناظر و AES للتشفير المتناظر.

قام الباحث بتوضيح البنية البرمجية المقترحة وكيفية توليد وتبادل المفاتيح بين المستخدمين، مع الاعتماد على آلية تشفير الرسائل بشكل منفصل لكل محادثة، بالإضافة إلى توثيق المستخدمين عبر رموز سرية.

أظهرت النتائج أن استخدام هذه التقنيات فعال نسبياً في حماية البيانات، لكن الدراسة أشارت إلى وجود صعوبات تقنية في إدارة المفاتيح وتوليدها بطريقة آمنة وسلسة بالنسبة للمستخدمين.

التحليل و المناقشة

الاستفادة الفعلية من هذه الدراسة ظهرت في أنها سلطت الضوء على مشاكل التشفير التقليدي مثل إدارة المفاتيح وعبء العمليات الحسابية الثقيلة لخوارزميات مثل RSA. دفعتنا هذه النتائج للبحث عن بروتوكولات أكثر كفاءة ومرونة، مع اعتماد خوارزميات تشفير قوية وعملية في نفس الوقت، وضرورة مراعاة سهولة الاستخدام للمستخدم النهائي.

٣. End-to-End Encrypted Messaging Protocols: An Overview

الملخص

تقدم هذه الدراسة مراجعة شاملة للبروتوكولات الأساسية المستخدمة في تطبيقات الدردشة والبريد الإلكتروني الآمنة، مع تركيز خاص على تقنيات التشفير من طرف إلى طرف. تستعرض الدراسة أكثر من ٣٠ مشروعاً ونظام ترسل آمن عالمي، وتناقش كيف تطورت الحلول من البروتوكولات التقليدية إلى البروتوكولات الحديثة مثل Signal و OTR و PGP و Matrix.

من أهم ما ركزت عليه الدراسة هو التحديات العملية التي تواجه المطورين، مثل إدارة المفاتيح وقابلية الاستخدام، وأهمية أن لا يتمكن الخادم أو أي جهة وسيطة من معرفة محتوى الرسائل. كما وضحت سبب انتشار بروتوكولات مثل Signal بسبب قوة أمانها وسهولة دمجها في التطبيقات العملية.

أما بالنسبة لهذا المشروع، فقد ساعدت هذه الدراسة بشكل مباشر في تكوين تصور واضح حول نقاط القوة والضعف في البروتوكولات الشائعة، وأكدت أهمية الاعتماد على بروتوكولات معتمدة وقوية، مثل Signal، كأساس للنظام المقترح. كما استفدنا من تحليل الدراسة للتحديات العملية التي يجب الانتباه لها أثناء التنفيذ، خصوصًا من ناحية قابلية الاستخدام والأمان الفعلي للتطبيق.

٤. A Formal Security Analysis of the Signal Messaging Protocol

الملخص [٩]

يعد هذا البحث واحدًا من أكثر الدراسات الأكاديمية تأثيرًا في مجال أمن بروتوكولات التراسل الحديثة، حيث يقدم تحليلًا رسميًا ودقيقًا لبروتوكول Signal الذي أصبح لاحقًا مرجعًا لتطبيقات كبرى مثل WhatsApp و Messenger. هدف البحث هو تقديم إطار رياضي وتحليل نظري يثبت فعليًا الضمانات الأمنية التي يوفرها بروتوكول Signal، بما يشمل خوارزميتي X3DH و Double Ratchet. يعرض الباحثون في البداية بنية بروتوكول Signal بالتفصيل، مع شرح الخطوات العملية للمصافحة الأولية (X3DH) وتحديث المفاتيح المستمر (Double Ratchet).

النتائج الرئيسية

أثبتت الدراسة أن بروتوكول Signal يقدم خاصيتي السرية الأمامية (Forward Secrecy) وسرية ما بعد الاختراق (Post-Compromise Security)، أي حتى لو تم تسريب مفتاح جلسة أو اختراق جهاز، تبقى الرسائل السابقة والمستقبلية آمنة ما لم يتم اعتراضها في اللحظة نفسها.

وضحت الدراسة آلية اشتقاق المفاتيح عبر Double Ratchet، وكيف تضمن تحديث المفاتيح مع كل رسالة تقريبًا. أشارت النتائج أيضًا إلى وجود بعض التحديات أو الفجوات عند تطبيق البروتوكول في محادثات جماعية أو سيناريوهات أجهزة متعددة، إلا أن البنية العامة بقيت قوية في المحادثات الثنائية. الدراسة أيضًا أظهرت فعالية استخدام X3DH لتبادل المفاتيح بين طرفين غير متزامنين (ليسوا أونلاين في نفس الوقت)، وأكدت على أهمية وجود مصفوفة مفاتيح احتياطية (prekeys) لإتمام المصافحة الآمنة.

التحليل و المناقشة

البحث يقدم مبررًا علميًا قويًا لاختيار بروتوكول Signal كمرجعية في بناء تطبيقات التراسل الآمن الحديثة، ويوضح لماذا يُعتبر X3DH و Double Ratchet من أقوى الأدوات الحالية لضمان أمن المحادثة.

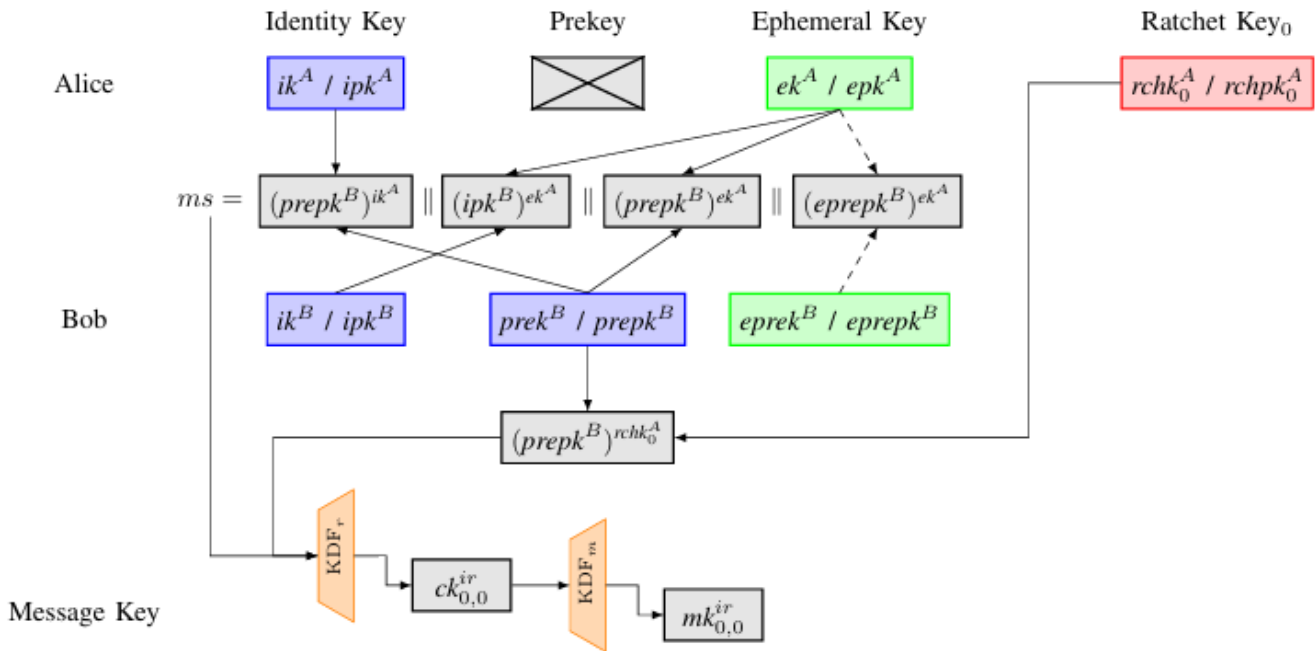
بناءً على نتائج الدراسة، فإن تطبيق Double Ratchet في المشروع يحقق سرية قوية ضد الاختراقات الدورية أو حتى في حال فقدان الجهاز أو تسريب أحد المفاتيح، لأن كل رسالة لها مفتاح مستقل مشتق من سلسلة مفاتيح تتغير باستمرار. كما أن الاعتماد على X3DH لتبادل المفاتيح يضمن إمكانية إنشاء جلسات آمنة حتى لو لم يكن الطرفان متصلين بنفس الوقت، ما يعزز مرونة وأمان التطبيق.

الشكل ٣,١ يوضح طريقة تبادل المفاتيح واشتقاق مفتاح الجلسة بشكل آمن من دون تدخل المخدم (Server) وهذا ما يحقق خاصية التشفير من طرف الى طرف في بروتوكول signal .

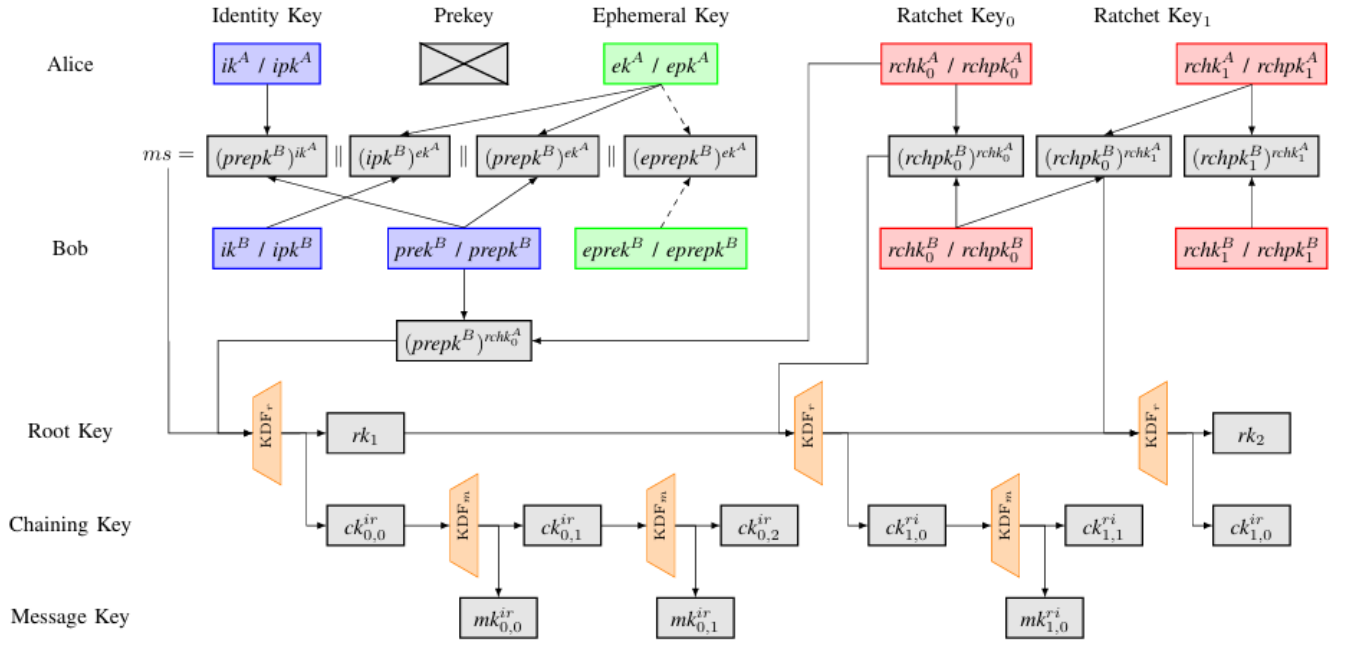
الشكل ٣,٢ يوضح عمل بروتوكول double ratchet بعد عملية تبادل المفاتيح و اشتقاق مفتاح الجلسة ، و الواضح من الشكل ان بروتوكول double ratchet يقوم بتشفير كل رسالة بفتاح فريد مشتق من مفتاح الجلسة وهذا ما يحقق خاصية السرية أمامية (Forward Secrecy) على مستوى ارسال الرسائل ويقوم بتغيير مفتاح الجذر واشتقاق مفاتيح جديدة لكل رسالة وهذا ما يحقق خاصية السرية ما بعد الاختراق (Post-Compromise Security)

الشكل ٣,٣ عبارة عن مثال مبسط لتدفق الرسالة بين طرفين A و B .

الجدول ٣,١ جدول يوضح الرموز المستخدمة في عملية X3DH و بروتوكول double ratchet .



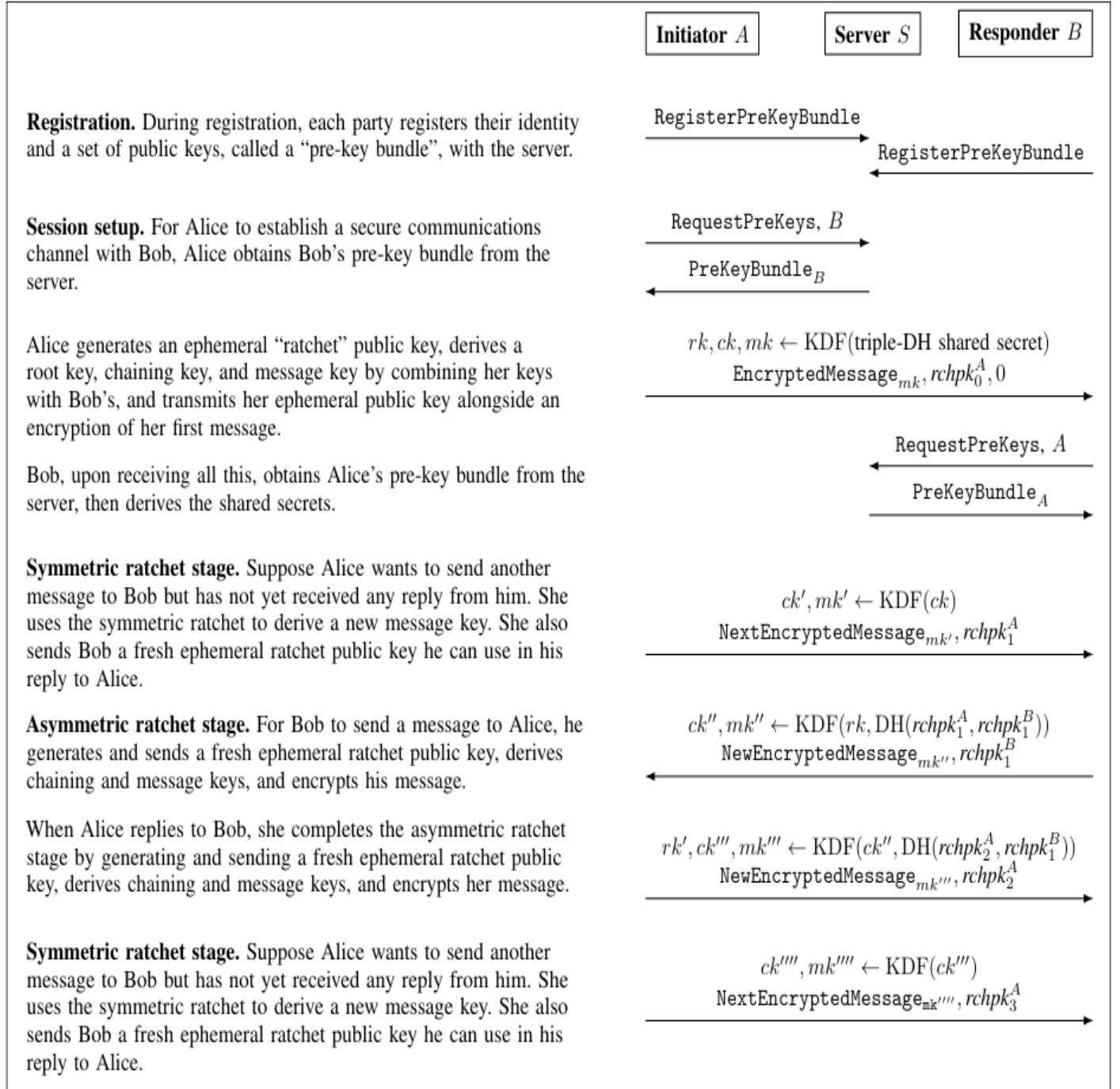
الشكل ٣,١ : يوضح عملية تبادل المفاتيح الأولي X3DH لبروتوكول signal



الشكل ٣، ٢: يوضح عملية تبادل مع المفاتيح و طريقة عمل بروتوكول double ratchet [٧]

٣، ١. جدول : يوضح الجدول رموز ازواج المفاتيح المتماثلة و غير المتماثلة المستخدمة في عملية تبادل المفاتيح و بروتوكول double ratchet [٧]

asymmetric	ipk^A	ik^A	A 's long-term identity key pair
	$prepk^B$	$prek^B$	B 's medium-term (signed) prekey pair
	$eprepk^B$	$eprek^B$	B 's ephemeral prekey pair (for the initial handshake)
	epk^A	ek^A	A 's ephemeral key pair (for the initial handshake)
	$rchpk_x^A$	$rchk_x^A$	A 's x^{th} ratchet key pair (for the x^{th} asymmetric ratchet)
symmetric		$ck_{x,y}^{ir}$	y^{th} chaining key in the x^{th} initiator–responder symmetric ratchet chain
		$ck_{x,y}^{ri}$	y^{th} chaining key in the x^{th} responder–initiator symmetric ratchet chain
		$mk_{x,y}^{ir}$	y^{th} message key in the x^{th} initiator–responder symmetric ratchet chain
		$mk_{x,y}^{ri}$	y^{th} message key in the x^{th} responder–initiator symmetric ratchet chain
		rk_x	x^{th} root key on the asymmetric ratchet



٣,٣. الشكل : يوضح تدفق رسالة مثال مبسط لتنفيذ بروتوكول Signal بين عميلين *A* و *B* عبر خادم *S* [٧]

٥. Breaking RSA encryption with a quantum computer

الملخص [١٠]

تستعرض هذه الدراسة الرائدة، المنشورة في مجلة Nature، تجربة عملية ناجحة لكسر نظام التشفير الكلاسيكي RSA باستخدام حاسوب كمومي. قام الباحثون بتطبيق خوارزمية شور الكمومية على حاسوب كمومي حقيقي، وتمكنوا من تحليل الأعداد الأولية الصغيرة التي يعتمد عليها نظام RSA، مما يمثل خطوة فارقة في تاريخ أمن التشفير. تؤكد النتائج أن القدرات الكمومية، رغم محدوديتها التقنية الحالية، تتطور بسرعة ويمكن أن تهدد أنظمة التشفير المنتشرة عالميًا في المستقبل القريب. بناءً على ذلك، تدعو الدراسة المجتمع العلمي إلى الإسراع في تطوير واعتماد خوارزميات تشفير مقاومة لكم للحفاظ على سرية المعلومات طويلة الأمد.

التحليل و المناقشة

نجح الفريق الباحث في إجراء تطبيق عملي لخوارزمية شور على حاسوب كمومي وتحليل أعداد أولية تشكل حجر الأساس لتشفير RSA. رغم أن الأرقام التي تم تحليلها ما تزال صغيرة (بسبب حدود التقنية الكمومية الحالية)، إلا أن إثبات المبدأ هذا يؤكد أن الأمر لم يعد مجرد نظرية بل واقع علمي متحقق.

تُبرز الدراسة أن تطوّر الحوسبة الكمومية لا يسير ببطء؛ إذ تشير التوقعات إلى أن الوصول لأجهزة قادرة على كسر مفاتيح RSA الكبيرة هو مسألة وقت. هذا يعني أن البيانات المشفرة اليوم قد تكون مهددة بالكسر مستقبلاً، خاصة إذا تم اعتراضها وتخزينها من قبل جهات تنتظر التقدم الكمومي ("collect now, decrypt later").

النتائج تضع أنظمة مثل RSA و ECC في دائرة الخطر. كل الأنظمة التي تعتمد في أمانها على صعوبة تحليل الأعداد الأولية أو اللوغاريتم المنفصل ستكون معرضة للكسر الكمومي. وبالتالي لا يمكن اعتبار هذه الأنظمة آمنة للأغراض التي تتطلب سرية طويلة الأمد (المراسلات الحكومية، المعلومات الطبية... إلخ).

تؤكد الدراسة ضرورة الانتقال العاجل إلى خوارزميات تشفير مقاومة لكم (Post-Quantum Cryptography)، وعدم الاكتفاء بالخوارزميات الكلاسيكية حتى لو كانت تُستخدم عالميًا اليوم. هذه الخوارزميات الجديدة (مثل Kyber) تعتمد على مشاكل رياضية لم تُكتشف بعد خوارزميات كمومية فعالة لحلها.

تشكل هذه الدراسة دعماً علمياً قوياً لاعتماد بروتوكولات هجينة أو مقاومة لكم من اليوم، إذ تبرر بوضوح أن التأمين ضد الهجمات الكمومية ليس خياراً ترفيهياً بل ضرورة للأنظمة الحساسة.

٦. National Institute of Standards and Technology. (2023). FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism (Kyber) – Draft

الملخص [١١]

تحدد هذه الوثيقة خوارزمية تشفير تُستخدم لإنشاء ونقل المفاتيح وتُعرف باسم Kyber. تُعد Kyber آلية تغليف مفاتيح (KEM) مبنية على الشبكات الرياضية (module-lattice)، وقد تم اختيارها من قبل NIST كمعيار لتوحيد التشفير ما بعد

الكم. صُممت Kyber لتوفر الأمان في مواجهة الحواسيب التقليدية والكمومية معًا، وتهدف إلى أن تكون بديلاً عن خوارزميات التشفير العامة المعرضة لهجمات كمومية.

تتميز هذه الخوارزمية بالكفاءة، ودعمها لعدة مستويات أمان، وملاءمتها لمجموعة واسعة من التطبيقات، بما في ذلك تأمين الاتصالات وإنشاء المفاتيح التشفيرية. تتناول الوثيقة شرح الخوارزمية، وأسباب اختيارها، والمعاملات الموصى بها، واعتبارات الأمان.

تحليل النتائج

■ أساس رياضي متين مقاوم للكم:

تم اختيار Kyber بناءً على مشكلة رياضية قوية في عالم الشبكات (lattice problems)، والتي يُعتقد أنها صعبة الحل حتى بواسطة الحواسيب الكمومية، بعكس خوارزميات مثل RSA أو ECC التي أصبحت معرضة للكسر باستخدام خوارزميات كمومية ك Shor's Algorithm.

■ الكفاءة والأداء:

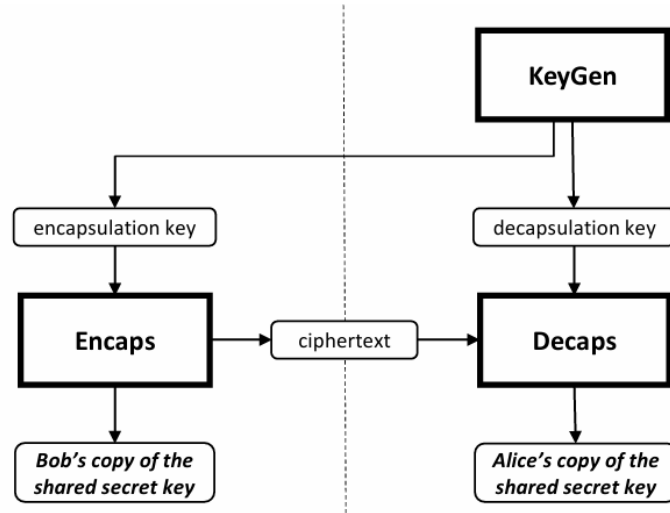
أظهرت الاختبارات أن Kyber تتميز بسرعة عالية في العمليات (توليد المفاتيح، التشفير، فك التشفير) وتستخدم حجم مفاتيح ورسائل أصغر من معظم مرشحات ما بعد الكم الأخرى، مما يجعلها عملية وسهلة الدمج في الأنظمة الحقيقية، سواء على الأجهزة المكتبية أو المحمولة.

■ مقارنة مع منافسين آخرين:

في مرحلة تقييم NIST، تفوقت Kyber من حيث الأداء، كفاءة الحجم، وتحليل الأمان على معظم المرشحات الأخرى، إذ واجهت بعض الخوارزميات الأخرى مشاكل في الاستقرار، البطء، أو صعوبة الدمج العملي.

■ اعتماد معياري رسمي:

اختيار NIST لخوارزمية Kyber لتكون الخوارزمية القياسية لتشفير المفاتيح في عصر ما بعد الكم يعطيها مصداقية عالمية ويوصي باعتمادها كخيار أول للحلول الأمنية المستقبلية، وهذا ما يجعل استخدامها في أي نظام ترسل أمن اليوم خيارًا مدعومًا علميًا ومؤسسيًا.



٣,٤. الشكل : رؤية بسيطة لإنشاء المفتاح باستخدام KEM [١١]

في مشروعنا تم تطبيق خوارزمية kyber في عملية تبادل المفاتيح بدمجها مع بروتوكول X3DH للحماية من الهجمات الكمومية و التقليدية

٧. الخلاصة

من خلال استعراض وتحليل الدراسات والأبحاث السابقة في مجال التشفير وتطبيقات التراسل الآمن، تبين أن التطورات المتسارعة في قدرات الحوسبة الكمومية باتت تشكل تهديداً واقعياً للأنظمة التقليدية المعتمدة مثل RSA و ECC. كما أظهرت الدراسات العملية والنظرية، ومن ضمنها دراسات منشورة في مجلات علمية رائدة مثل Nature وأبحاث NIST، أن مستقبل الأمان الرقمي يرتبط بشكل أساسي بمدى قدرة الأنظمة على تبني خوارزميات مقاومة للكم، مثل Kyber، والتي أثبتت فعاليتها وكفاءتها العملية من خلال تقييمات ومعايير عالمية.

علاوة على ذلك، أكدت دراسات تحليلية رسمية مثل بحث Cohn-Gordon وآخرين (٢٠١٧) حول بروتوكول Signal، أن دمج خوارزميات (Double Ratchet) وبروتوكولات تبادل المفاتيح الحديثة (X3DH) يوفر خصائص أمان متقدمة مثل السرية الأمامية وسرية ما بعد الاختراق والعديد من الخصائص الأمنية الأخرى، إلا أن هذه الأنظمة بحاجة إلى تحديث جذري لمقاومة التهديدات الكمومية المستقبلية.

بناءً على ما سبق، يتضح أن كل عمل بحثي جديد في مجال التشفير الآمن لا بد أن يأخذ بعين الاعتبار ضرورة مقاومة الهجمات الكمومية، مع الحفاظ على الأداء والمرونة العملية. وقد شكلت هذه النتائج أساساً علمياً متيناً لتطوير البروتوكول المقترح في هذا المشروع، والذي يهدف إلى تقديم نظام تراسل آمن يجمع بين أحدث المعايير الأمنية التقليدية والتوجهات المستقبلية في التشفير ما بعد الكم.

الفصل الرابع

تطوير نظام التشفير

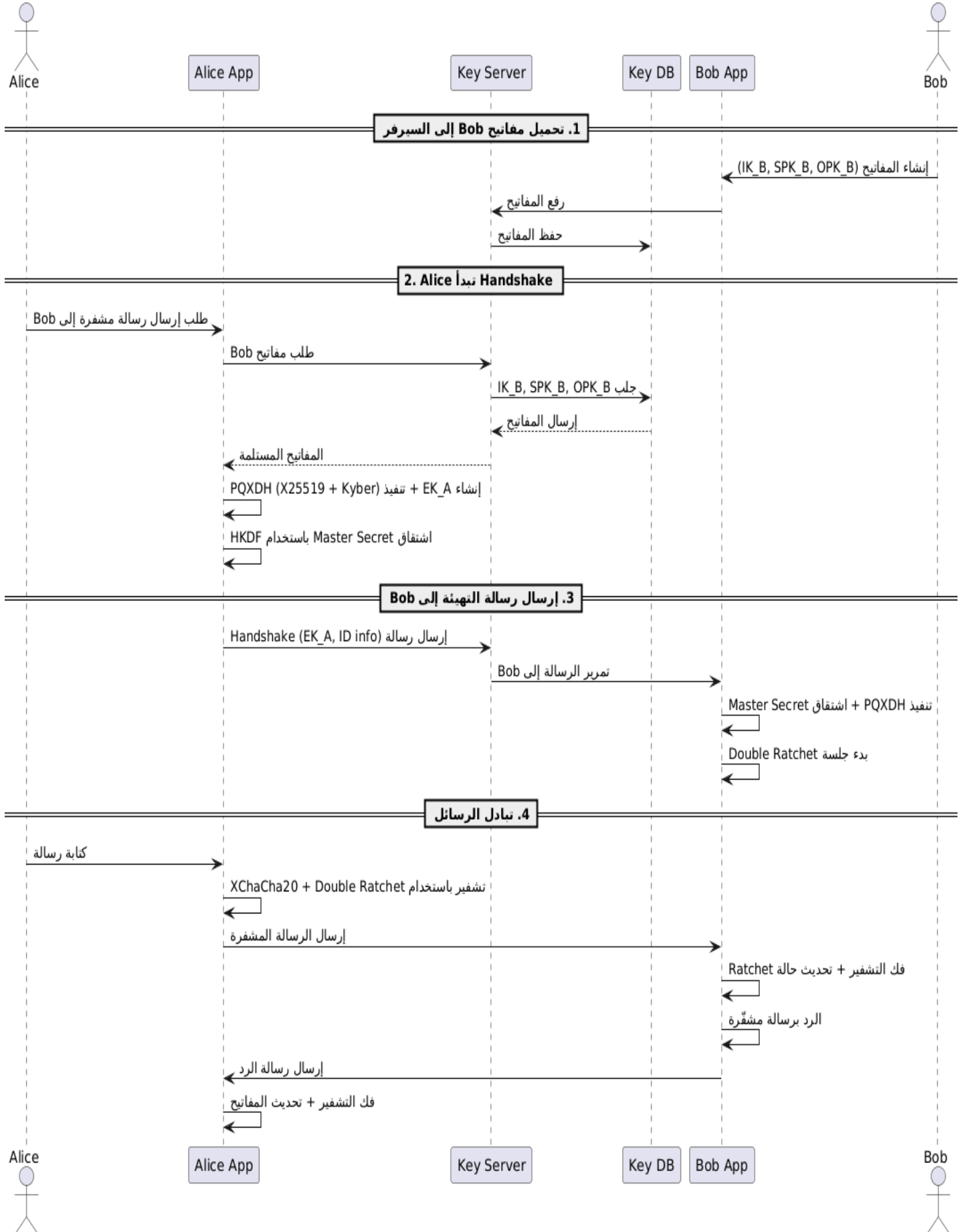
يعتمد بناء اي تطبيق لتبادل الرسائل على نظام التشفير الذي يؤمن عملية تبادل الرسائل بين المستخدمين بشكل امن ،
لا قيمة لأي تطبيق تبادل رسائل اذا لم يكن امن .

اخترنا في هذا المشروع العمل على نظام تشفير يرتكز على اهم خوارزميات التشفير المتقدمة والمثبت عليها قوتها و
مرونتها و ايضا سرعتها ،

يركز هذا الفصل على تقديم صورة شاملة لبنية النظام البرمجية والوظيفية، مع توضيح كيفية دمج البروتوكولات
التشفيرية المتقدمة ضمن سير العمل للنظام.

إضافة إلى شرح أسباب اختيار كل مكوّن أو تقنية وتبرير ذلك علمياً وعملياً. كما سيتم استعراض تدفق البيانات، مراحل
إدارة الجلسات، وآليات حماية المفاتيح والرسائل، بهدف نقل القارئ من الجانب النظري إلى الواقع العملي لتطوير النظام
المقترح.

٢. المخطط العام لنظام التشفير المستخدم



١. مقدمة عن المخطط

يمثل المخطط التوضيحي ١، ٤ المرفق سير عملية تشفير وتبادل الرسائل النصية بين طرفين (Alice) و (Bob) في نظام التراسل الآمن، مع الاعتماد على بروتوكول هجين يجمع تقنيات X3DH و Kyber (في تبادل المفاتيح)، و Double Ratchet (في تحديث مفاتيح الرسائل بشكل مستمر)، وتشفير الرسائل الفعلية باستخدام XChaCha20-Poly1305.

٢. شرح مكونات النظام

١. تطبيق العميل (Client App)

هو البرنامج الذي يستخدمه كل مستخدم نهائي على جهازه

٢. الخادم (Key Server / Server)

هو الوسيط الذي يدير عملية تسجيل وتبادل المفاتيح ورسائل التراسل، لكنه لا يطلع على أي معلومات حساسة أو مفاتيح سرية.

٣. قاعدة بيانات المفاتيح (Key Database)

هي قاعدة بيانات يتم فيها تخزين المفاتيح العامة المؤقتة الخاصة بكل مستخدم.

٤. وحدة إدارة المفاتيح والبروتوكولات (Key & Protocols Engine)

هذه الوحدة تكون مدمجة داخل تطبيق العميل (Client)، وتعتبر القلب التشفيري للنظام.

٣. خوارزميات التشفير المستخدمة في النظام

- X25519 – لتبادل المفاتيح الكلاسيكي
- Kyber – لتبادل المفاتيح المقاوم للهجمات الكمومية
- HKDF – اشتقاق المفاتيح
- Double Ratchet – إدارة المفاتيح الديناميكية
- XChaCha20-Poly1305 – تشفير الرسائل

شرح مراحل عمل البروتوكول المطور

٤. المرحلة الأولى : إعداد المفاتيح ورفعها إلى الخادم

تمثل هذه المرحلة الخطوة التأسيسية في بناء الأمن داخل النظام، حيث يتم توليد المفاتيح التشفيرية الضرورية لكل مستخدم لحظة تسجيله في التطبيق. وتشمل هذه المفاتيح ما يضمن سرية الاتصال، المصادقة، والحماية من الهجمات المستقبلية، بما في ذلك الهجمات الكمومية.

تسمح هذه المرحلة من التواصل مع مستخدم بشكل امن حتى لو لم يكن متصل .

١. توليد المفاتيح على جهاز المستخدم :

عند تسجيل مستخدم جديد ، يقوم النظام بتوليد مجموعة من المفاتيح اللازمة لإجراء المصادقة وتبادل الرسائل الآمنة. كل مستخدم يولد محلياً ما يلي :

١,١. مفاتيح X25519 (لتأمين التشفير الكلاسيكي):

▪ مفتاح الهوية (Identity Key - IK):

مفتاح دائم يستخدم للتعريف بهوية المستخدم.

▪ مفتاح موقع مسبقاً (Signed Prekey - SPK):

مفتاح متوسط الأجل، يتم توقيعه باستخدام مفتاح الهوية لضمان أصالته.

▪ مجموعة مفاتيح أحادية الاستخدام (One-Time Prekeys - OPKs):

تولد بعدد محدد (مثلاً ١٠٠ مفتاح)، وتُستخدم لمرة واحدة في المصادقة لتوفير سرية أمامية قوية.

١,٢. مفاتيح Kyber (لتأمين ما بعد الكم):

زوج مفاتيح Kyber (عام + خاص):

▪ المفتاح العام يُستخدم ضمن المصادقة PQXDH.

▪ المفتاح الخاص يُخزن محلياً في جهاز المستخدم.

▪ هذا المفتاح لا يُستخدم في التشفير المباشر، بل يُستخدم لتغليف سر الجلسة (encapsulate)

(shared secret) من قبل الطرف الآخر.

٢. رفع المفاتيح العامة إلى الخادم :

يتم إرسال الحزمة التالية إلى الخادم ليتم تخزينها في قاعدة بيانات المخدم :

نوع المفتاح	يستخدم
IK_pub	التحقق من الهوية
SPK_pub	حماية المصافحة
OPK_pub[]	مفتاح واحد لكل جلسة جديدة
Kyber_pub	تغليف السر المشترك المقاوم للكم

٥.

٦. المرحلة الثانية: المصافحة الأولية وبناء الجلسة (Initial Handshake and Session Establishment)

تهدف هذه المرحلة إلى تمكين طرفين (مثلاً: Alice و Bob) من إنشاء سر مشترك بطريقة آمنة، يمكن استخدامه لاحقاً كأساس لجلسة مشفرة باستخدام خوارزمية Double Ratchet.

وتُعد هذه الخطوة محورية في تحقيق سرية الطرفين من البداية، حتى في حال عدم اتصالهما في الوقت ذاته، وهي مصممة لتوفير كل من:

- السرية الأمامية (Forward Secrecy)
- المصادقة (Authentication)
- مقاومة الهجمات الكمومية (Post-Quantum Security)

ويتم تنفيذ هذه المصافحة من خلال بروتوكول PQXDH وهو بروتوكول هجين يجمع بين بروتوكول X3DH الكلاسيكي وخوارزمية Kyber المقاومة للكم.

تتم عملية المصافحة الأولية وبناء الجلسة بهذه الخطوات :

١- استرجاع حزمة المفاتيح (prekey bundle) من الخادم :

عندما تقرر Alice إرسال رسالة إلى Bob لأول مرة، فإنها تطلب من الخادم حزمة المفاتيح العامة الخاصة بـ Bob أي أنها تطلب المفاتيح الموجود في الجدول ٤,١ جدول prekey bundle

يتم اختيار مفتاح واحد من OPK-B (إن وُجد) ليتم استهلاكه في هذه الجلسة، ويقوم الخادم بإزالته بعد تسليمه.

٢- توليد المفاتيح المؤقتة وتنفيذ المصافحة على جهاز Alice:

على جانب Alice، يتم توليد مفتاح مؤقت جديد EK_A (ephemeral key) باستخدام X25519، ويُستخدم مع مفاتيح Bob المسترجعة لتوليد سر الجلسة الرئيسي (Master Secret).

العمليات التي تنفذها Alice:

٢,١ عمليات تبادل المفاتيح باستخدام X25519 (كما في X3DH في الشكل ٣,١ ص ٢١) لكن بشكل مبسط :

- $DH1 = DH(IK_A, SPK_B)$
- $DH2 = DH(EK_A, IK_B)$
- $DH3 = DH(EK_A, SPK_B)$
- $DH4 = DH(EK_A, OPK_B) \leftarrow$ إن وُجد

٢,٢ تغليف مفتاح Kyber:

تقوم Alice بتغليف قيمة سرية باستخدام Kyber_pub_B $\leftarrow$ الناتج: Shared_Kyber_Secret

٢,٣ مزج الأسرار باستخدام HKDF:

يتم دمج جميع نتائج DH مع Shared_Kyber_Secret في دالة مشتقة المفاتيح HKDF لإنتاج:

$$\text{Master Secret} = \text{HKDF}(DH1 || DH2 || DH3 || DH4 || \text{Shared_Kyber_Secret})$$

هذا السر Master Secret هو الجذر الأمني الذي ستبنى عليه جلسة Double Ratchet لاحقًا.

٣- إرسال أول رسالة إلى Bob:

تقوم Alice بإرسال أول رسالة إلى Bob عبر الخادم، وتُسمى هذه الرسالة بـ Message 0.

تتضمن الرسالة ما يلي:

- المفتاح المؤقت EK_A
- المعرف العام لـ Alice (أو بصمتها المفتاحية)
- تغليف Kyber الناتج (Kyber ciphertext)
- نص مشفر باستخدام مفتاح مشتق من Master Secret

ملاحظة : الرسالة تمر عبر الخادم، لكن لا يمكنه قراءتها أو الوصول إلى أي معلومات حساسة بداخلها.

٤- استلام الرسالة وبناء الجلسة على جهاز Bob:

عند استلام Bob للرسالة الأولى من Alice، يقوم بما يلي:

العمليات على جهاز Bob:

يستخدم EK_A، مفاتيحه الخاصة، وتغليف Kyber لفك نفس السر المغلف.

ينفذ نفس عمليات DH التي نفذتها Alice مع الدخل التالي :

- $DH1 = DH(IK_A_pub, SPK_B_priv)$
- $DH2 = DH(EK_A_pub, IK_B_priv)$
- $DH3 = DH(EK_A_pub, SPK_B_priv)$
- $DH4 = DH(EK_A_pub, OPK_B_priv)$ ← إذا وجد

يستخرج نفس Shared_Kyber_Secret باستخدام مفتاحه الخاص.

يُدخل جميع القيم في HKDF ليحصل على نفس Master Secret كما عند Alice.

وهكذا تتطابق المفاتيح السرية على كلا الطرفين دون أن يكون أي طرف ثالث (بما في ذلك الخادم) قادرًا على حسابها.

٥- بدء سلسلة المفاتيح (Double Ratchet):

بعد إتمام المصافحة بنجاح، يبدأ الطرفان تنفيذ خوارزمية Double Ratchet، كما في الشكل ٣,١ ص ٢١ التي تعتمد على:

- Root Key ← مشتقة من Master Secret
- Chain Key ← لتوليد مفاتيح كل رسالة على حدة
- DH Ratchet ← لتجديد المفتاح بعد فترات معينة (غير مطبقة حاليًا في النظام)

❏ تؤسس الجلسة الأمنية من هذه النقطة، ويتم تحديث المفاتيح تلقائيًا بعد كل رسالة.

٧. المرحلة الثالثة: بدء جلسة Double Ratchet وتبادل الرسائل

بعد أن تنتهي المصافحة الأولية بنجاح ويصل الطرفان إلى Master Secret مشترك، يبدأ بروتوكول Double Ratchet بالعمل، وهو المسؤول عن تأمين تبادل الرسائل بشكل مستمر وديناميكي.

هذا البروتوكول هو ما يمنح النظام خصائص أمنية متقدمة مثل:

- السرية الأمامية على مستوى الرسائل (Forward Secrecy): لا يمكن كشف الرسائل السابقة حتى في حال اختراق الجهاز لاحقًا.
- السرية ما بعد الاختراق (Post-Compromise Security): إذا اخترق الجهاز ثم تم تأمينه، يعود النظام تدريجيًا إلى حالة أمان.

- مفتاح جديد لكل رسالة (Per-Message Keying): كل رسالة يتم تشفيرها وفكها بمفتاح فريد لا يُعاد استخدامه.

١- نظرة عامة على Double Ratchet

يتكوّن بروتوكول Double Ratchet من:

:DH Ratchet

يُحدث مفتاح الجذر (Root Key) كلما تم توليد مفتاح DH جديد.

Chain Ratchets (الإرسال والاستقبال):

سلاسل مستقلة تستخدم لتوليد مفاتيح الرسائل بناءً على مفتاح سلسلة واحد (Chain Key).

دالة اشتقاق المفاتيح (HKDF):

مسؤولة عن اشتقاق المفاتيح من السلاسل وإنتاج مفاتيح كل رسالة.

٢- تهيئة الجلسة بعد المصافحة تتم تماماً كما في الشكل ٣,٢ ص ٢٢ ولكن بشكل مبسط :

- يتم استخدام Master Secret كمفتاح جذر أولي RK_0 .
- يتم إنشاء سلسلة إرسال CK_s وسلسلة استقبال CK_r باستخدام HKDF.

كل سلسلة تنشئ مفاتيح رسائل متسلسلة:

$$Message_Key_1 = HKDF(CK_0)$$

$$Message_Key_2 = HKDF(CK_1)$$

...

٣- إرسال رسالة من Alice إلى (Bob Message 1)

٣,١ توليد مفتاح الرسالة:

قبل إرسال الرسالة، تقوم Alice بتحديث سلسلة الإرسال:

$$CK_s, MK = HKDF(CK_s)$$

- CK_s : سلسلة الإرسال (تُحدث)
- MK: مفتاح تشفير الرسالة

٣,٢ تشفير الرسالة:

يتم تشفير النص باستخدام XChaCha20-Poly1305 مع المفتاح MK.

$$Ciphertext = Encrypt(MK, plaintext, header)$$

٣,٣ إرسال الرسالة:

يتم إرسال الرسالة المشفرة إلى الخادم.

الخادم لا يفك تشفيرها، بل يمررها كما هي إلى Bob.

٤ - استلام Bob للرسالة (Message 1)

٤,١ التحقق من نوع الرسالة:

إذا كانت الرسالة تحتوي مفتاح DH جديد من Alice، ينفذ DH Ratchet.

٤,٢ تحديث سلسلة الاستقبال:

Bob يستخدم CK_r لاشتقاق مفتاح الرسالة:

$$CK_r, MK = HKDF(CK_r)$$

٤,٣ فك التشفير:

يستخدم Bob MK لفك تشفير النص باستخدام نفس خوارزمية التشفير المتناظر Xchacha20-poly-1305.

في هذا الفصل، تم تقديم عرض تفصيلي للبنية المعمارية و الوظيفة للنظام المقترح، بدءًا من تحليل مكوناته الأساسية، ومرورًا بشرح متسلسل لجميع مراحل العمل التشفيري، من توليد المفاتيح الأولية، إلى المصافحة الآمنة باستخدام بروتوكول PQXDH، وانتهاءً بتبادل الرسائل ضمن جلسة Double Ratchet المتطورة.

تمّ الاعتماد على بنية هجينة تجمع بين خوارزميات تقليدية موثوقة مثل X25519، وخوارزميات مقاومة للهجمات الكمومية مثل Kyber، إضافة إلى تطبيق دالة اشتقاق المفاتيح HKDF لضمان إنتاج مفاتيح فرعية آمنة. وقد مكّنت هذه البنية من تحقيق خصائص أمان عالية المستوى، أهمها:

- السرية الأمامية (Forward Secrecy): كل رسالة مشفرة بمفتاح فريد غير قابل لإعادة الاستخدام.
- السرية ما بعد الاختراق (Post-Compromise Security): القدرة على استعادة الأمان حتى بعد تسريب مؤقت للمفاتيح.
- مقاومة الهجمات الكمومية: من خلال دمج Kyber ضمن المصافحة وتأمين الجلسة ضد الكسر المستقبلي.
- عدم الاعتماد على الخادم: حيث لا يمتلك الخادم أي معرفة بالمفاتيح الخاصة أو محتوى الرسائل.

كما تم توثيق جميع العمليات التشفيرية بتفصيل دقيق، بما يشمل تسلسل توليد المفاتيح، استخدام HKDF، وتحديث الجلسة عبر Double Ratchet، بما يضمن بنية واضحة وقابلة للتنفيذ العملي.

الفصل الخامس

التطبيق العملي و النتائج

تم خلال هذا المشروع تصميم واجهة تطبيق بسيطة تعتمد على سطر الأوامر (Terminal-based)، بهدف إثبات فعالية البروتوكول الأمني المصمم، وتنفيذ جميع مراحل التشفير والتبادل التلقائي للمفاتيح بين الأطراف بنجاح.

وقد ركّز في هذه المرحلة على بناء البنية التحتية التشفيرية للنظام بشكل دقيق وآمن، وضمان عمل جميع المكونات البرمجية الأساسية مثل توليد المفاتيح، المصافحة الهجينة (PQXDH)، إدارة جلسات Double Ratchet، وتبادل الرسائل المشفرة.

بالرغم من بساطة الواجهة الحالية، إلا أنها تحقق الوظائف المطلوبة بدقة، وتوفر بيئة اختبار واضحة لجميع السيناريوهات الأمنية الممكنة.

بيئة التطوير والأدوات المستخدمة

١ - لغة البرمجة :

تم تطوير النظام باستخدام لغة Python 3.10، نظرًا لما توفره من مكتبات قوية في مجال التشفير، ودعمها الجيد لخوارزميات ما بعد الكم والأدوات منخفضة المستوى اللازمة لتنفيذ البروتوكولات المعقدة مثل X3DH و Kyber و Double Ratchet.

٢ - بيئة العمل:

نظام التشغيل: تم تنفيذ النظام واختباره على بيئة كالي لينكس (Kali Linux) الافتراضية، نظرًا لكونها بيئة مهيأة مسبقًا لأغراض الأمن السيبراني والتشفير، وتوفر أدوات متقدمة لدعم العمل في مجال البروتوكولات الآمنة.

بيئة التطوير (IDE): تم استخدام text editor ضمن بيئة Kali، مع دعم Python وبيئة افتراضية (venv) لإدارة التبعيات والمكتبات المطلوبة بشكل معزول وآمن.

٣- المكتبات والأطر المستخدمة:

- **oqs** : تنفيذ خوارزمية Kyber1024 لتشفير ما بعد الكم
- **Nacl** : تشفير الرسائل باستخدام XChaCha20-Poly1305
- **Pycryptodome** : توليد المفاتيح البيضاوية وتوقيع الرسائل
- **socket, threading** : (في النسخ التجريبية الأولى) إدارة الاتصالات الشبكية يدويًا
- **Flask** : إطار خفيف لتوفير واجهة خادم محلي (API) بين العميل والخادم
- **pickle, base64** : ترميز وتسلسل البيانات بين العميل والخادم

٤- هيكلية المشروع البرمجية:

- **client.py** يمثل جهة المستخدم : توليد المفاتيح، تنفيذ المصافحة، إرسال واستقبال الرسائل
- **server.py** واجهة الخادم المبنية باستخدام Flask: استقبال حزم المفاتيح وتمرير الرسائل
- **pqx3dh.py** تنفيذ بروتوكول المصافحة الهجينة PQXDH
- **ratchet.py** تطبيق خوارزمية Double Ratchet لإدارة المفاتيح التلقائية
- **crypto_utils.py** العمليات التشفيرية الأساسية: X25519, Kyber, HKDF، التوقيع، التشفير

توليد وتخزين مفاتيح المستخدم (User Key Generation):

عند تسجيل مستخدم جديد، يقوم النظام تلقائياً بتوليد مفاتيح الهوية (Identity Key IK)، المفتاح المسبق الموقع (Signed PreKey SPK)، مجموعة مفاتيح مؤقتة (OPK)، ومفاتيح Kyber العامة والخاصة كما في الشكل ٥,١

```
class UserKeys:
    def __init__(self, username):
        self.IK = ecc_generate() # ECC Identity Key
        self.SPK = ecc_generate() # ECC Signed PreKey
        self.SPK_sig = sign(self.IK, ecc_pub_bytes(self.SPK))
        self.OPKs = [ecc_generate() for _ in range(10)]
        self.kyber_pub, self.kyber_priv = kyber_generate()

    def get_prekey_bundle(self):
        return {
            "IK": ecc_pub_bytes(self.IK),
            "SPK": ecc_pub_bytes(self.SPK),
            "SPK_sig": self.SPK_sig,
            "OPK": ecc_pub_bytes(self.OPKs[0]),
            "OPK_id": 0,
            "kyber_pub": self.kyber_pub
        }
```

٥,١. الشكل : كود توليد المفاتيح

يتم تخزين المفاتيح العامة فقط في قاعدة البيانات، والمفاتيح الخاصة تُحفظ في الذاكرة أو ملف مشفر محلياً.

إنشاء الجلسة (Session Handshake)

عند بدء المحادثة بين مستخدمين، يحصل الطرف المرسل (Alice) على pre-key bundle للطرف المستقبل (Bob)، ثم ينفذ بروتوكول تبادل المفاتيح الهجين الذي يجمع X3DH و Kyber.

```
def pqx3dh_initiate(my_ik, prekey_bundle):
    EK = ecc_generate()
    IK_B = ecc_from_bytes(prekey_bundle["IK"])
    SPK_B = ecc_from_bytes(prekey_bundle["SPK"])
    SPK_sig = prekey_bundle["SPK_sig"]
    OPK_B = ecc_from_bytes(prekey_bundle["OPK"]) if prekey_bundle["OPK"] else None
    kyber_B_pub = prekey_bundle["kyber_pub"]

    # التأكد من صحة التوقيع الرقمي للمفتاح المسمى
    if not verify(IK_B.public_key(), ecc_pub_bytes(SPK_B), SPK_sig):
        raise Exception("Signed PreKey signature invalid!")

    # الكلاسيكية DHs حساب
    DH1 = dh(my_ik, SPK_B)
    DH2 = dh(EK, IK_B)
    DH3 = dh(EK, SPK_B)
    DH4 = dh(EK, OPK_B) if OPK_B else b''

    # تنفيذ Kyber PQC لتغليف مفتاح
    kyber_shared_secret, kyber_ct = kyber_encapsulate(kyber_B_pub)

    # استنتاج مفتاح الجلسة النهائي
    ms = DH1 + DH2 + DH3 + DH4 + kyber_shared_secret
    SK = hkdf(b"PQX3DH", ms, b"pqx3dh_protocol", 32)

    return SK, EK, kyber_ct, None # مفتاح مؤقت خاص بـ EK
```

٥,٢. الشكل : كود إنشاء الجلسة

تُرسل الرسالة الأولى مع (IK_A, EK_A, kyber_ct) لبوب.

استقبال الرسالة الأولى وإنشاء الجلسة (Responder)

عندما يستقبل Bob الرسالة الأولى، ينفذ نفس العمليات، لكن يستخدم مفتاحه الخاص لفك تغليف kyber_ct واسترجاع السر المشترك

```
def pqx3dh_respond(my_keys, msg_info):
    IK_A = ecc_from_bytes(msg_info["IK_A"])
    EK = ecc_from_bytes(msg_info["EK_A"])
    SPK = my_keys.SPK
    IK_B = my_keys.IK
    OPK = my_keys.get_opk_obj(msg_info["OPK_id"])
    kyber_ct = msg_info["kyber_ct"]

    kyber_shared_secret = kyber_decapsulate(my_keys.kyber_priv, kyber_ct)

    DH1 = dh(SPK, IK_A)
    DH2 = dh(IK_B, EK)
    DH3 = dh(SPK, EK)
    DH4 = dh(OPK, EK) if OPK else b''

    ms = DH1 + DH2 + DH3 + DH4 + kyber_shared_secret
    SK = hkdf(b"PQX3DH", ms, b"pqx3dh_protocol", 32)

    # على نفس الحالة Double Ratchet تهيئة
    ratchet = DoubleRatchet(SK, my_ecc=SPK, their_pub=EK)
    return SK, ratchet
```

٥,٣. الشكل : كود انشاء الجلسة من طرف المستقبل

تشفير وفك تشفير الرسائل (Double Ratchet + XChaCha20)

كل رسالة جديدة ضمن الجلسة تُشفّر بمفتاح مشتق فريد باستخدام Double Ratchet ، ويتم تشفير الرسالة عن طريق خوارزمية XChaCha20-Poly1305.

```
class DoubleRatchet:
    def __init__(self, root_key, my_ecc=None, their_pub=None):
        self.DHs = my_ecc or ecc_generate()
        self.DHr = their_pub
        self.root_key = root_key
        self.sending_chain_key = None
        self.receiving_chain_key = None
        self.sending_message_num = 0
        self.receiving_message_num = 0
        if their_pub is not None:
            self._initialize_chains()

    def _initialize_chains(self):
        dh_shared = dh(self.DHs, self.DHr)
        rk_ck = hkdf(self.root_key, dh_shared, b"root", 64)
        self.root_key = rk_ck[:32]
        self.sending_chain_key = rk_ck[32:]
        self.receiving_chain_key = None

    def encrypt(self, plaintext):
        if self.sending_chain_key is None:
            raise Exception("No sending chain key!")
        message_key = hkdf(self.sending_chain_key, b"0", b"message", 32)
        self.sending_chain_key = hkdf(self.sending_chain_key, b"1", b"chain", 32)
        self.sending_message_num += 1
        ct = xchacha20_encrypt(message_key, plaintext)
        pub_bytes = ecc_pub_bytes(self.DHs)
        return pub_bytes, ct

    def decrypt(self, pub_bytes, ciphertext):
        their_pub = ecc_from_bytes(pub_bytes)
        if self.DHr is None or pub_bytes != ecc_pub_bytes(self.DHr):
            self.dh_ratchet(their_pub)
        if self.receiving_chain_key is None:
            raise Exception("No receiving chain key!")
        message_key = hkdf(self.receiving_chain_key, b"0", b"message", 32)
        self.receiving_chain_key = hkdf(self.receiving_chain_key, b"1", b"chain", 32)
        self.receiving_message_num += 1
        pt = xchacha20_decrypt(message_key, ciphertext)
        return pt.decode() if isinstance(pt, bytes) else pt
```

٥,٤. الشكل : كود التشفير وفك التشفير

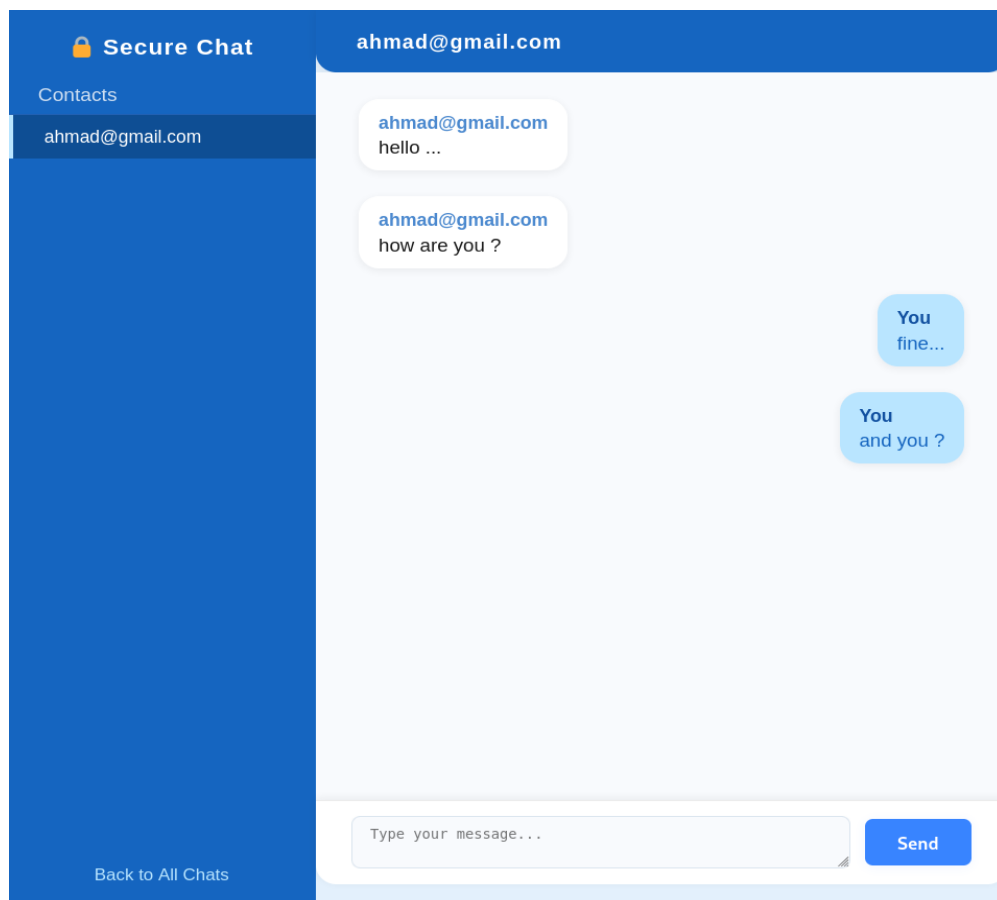
تُخزن الرسائل فور إرسالها بشكل مشفر بالكامل كما في الشكل ٥,٥ :

```
msg = Message(  
    sender=sender,  
    recipient=recipient,  
    pub_bytes=pub_bytes,  
    ciphertext=ct,  
    is_initial=is_initial,  
    IK_pub=ecc_pub_bytes(keys.IK),  
    kyber_ct=kyber_ct,  
    kyber_eph_pub=kyber_eph_pub  
)  
db.session.add(msg)  
db.session.commit()
```

٥,٥. الشكل : كود تخزين الرسائل

واجهة المستخدم وعرض الرسائل :

تم تطوير واجهة ويب باستخدام (HTML/CSS/JS)، تعرض الرسائل غير قابلة للقراءة إلا من قبل الطرف المعني، وتسمح بالتبادل الفوري مع حماية حقيقية على مستوى كل رسالة.



٥,٦. الشكل : واجهة الدردشة في التطبيق

النتائج والمناقشة

أظهر التطبيق العملي للبروتوكول الأمني المصمم فاعلية واضحة في تحقيق الأهداف الأمنية الأساسية للنظام، وذلك من خلال تنفيذ المصافحة الهجينة PQXDH وتكاملها السلس مع خوارزمية Double Ratchet لتشفير الرسائل.

نجحت آلية تبادل المفاتيح في توليد سر مشترك بين الطرفين دون الحاجة إلى تزامن مباشر، وتمكّن النظام من بناء جلسة آمنة بالكامل باستخدام مفاتيح ديناميكية تتجدد بعد كل رسالة.

أبرز الخصائص الأمنية التي تم تحقيقها عملياً تشمل:

- السرية من طرف إلى طرف (End-to-End Encryption):

حيث تم التأكد من أن جميع الرسائل يتم تشفيرها وفك تشفيرها فقط على أطراف المحادثة، دون أن يمتلك الخادم القدرة على قراءة أي محتوى.

- السرية الأمامية (Forward Secrecy):

إذ إن كل رسالة تستخدم مفتاحاً فريداً مشتقاً من سلسلة مفاتيح متغيرة، مما يمنع فك الرسائل السابقة حتى لو تم الوصول إلى المفاتيح لاحقاً.

- السرية ما بعد الاختراق (Post-Compromise Security):

بفضل بروتوكول Double Ratchet، يستعيد النظام أمانه تدريجياً حتى لو تم اختراق أحد الأطراف مؤقتاً.

- مقاومة الهجمات الكمومية:

تم دمج خوارزمية Kyber بنجاح ضمن المصافحة لتوفير طبقة أمان ضد هجمات الحوسبة الكمومية، بما يتماشى مع توصيات NIST والدراسات المرجعية المعتمدة.

- بساطة التنفيذ وقابلية التوسع:

رغم أن الواجهة الحالية بسيطة وتعتمد على الطرفية، إلا أن التصميم الداخلي للنظام مرّن وقابل للتوسعة بسهولة ليشمل واجهات أكثر تطوراً وخدمات إضافية مثل إرسال الملفات أو دعم عدة أجهزة.

تناولت هذه الدراسة تصميم وتنفيذ نظام مراسلة آمن يعتمد على أحدث المبادئ التشفيرية، مع التركيز على مقاومة التهديدات الحالية والمستقبلية، وخاصة تلك المرتبطة بالحواسيب الكمومية.

من خلال مراجعة الإطار النظري والدراسات السابقة، تم تحديد نقاط الضعف في البروتوكولات التقليدية مثل X3DH، الأمر الذي استدعى تطوير بروتوكول مصافحة هجين (PQXDH) يجمع بين خوارزميات التشفير البيضاوي والتشفير ما بعد الكم (Kyber) لضمان سرية مستقبلية طويلة الأمد.

كما تم دمج بروتوكول Double Ratchet ضمن النظام لتوفير سرية أمامية وما بعد الاختراق، مما يعزز من قوة النموذج المقترح ويجعل كل رسالة مشفرة بمفتاح مستقل غير قابل لإعادة الاستخدام.

وقد تم اختبار النظام ضمن بيئة كالي لينكس باستخدام لغة Python ومجموعة من المكتبات المتخصصة، مع تطوير واجهة أولية بسيطة لإثبات فعالية البروتوكول في بيئة عملية حقيقية.

أظهرت نتائج التطبيق أن النظام قادر على تحقيق الخصائص الأمنية المرجوة بشكل متكامل، بما يشمل:

- التشفير من طرف إلى طرف
- سرية المفاتيح
- ديناميكية تحديث الجلسات
- مقاومة الهجمات الكمومية
- وذلك دون التضحية ببساطة التصميم أو كفاءة الأداء.

في الختام، يمثل هذا المشروع خطوة عملية أولى نحو بناء تطبيقات ترسل أكثر أماناً، تواكب تحديات المستقبل التشفيري، وتمنح المستخدمين الثقة الكاملة في سرية تواصلهم الرقمي.

1. Abdalla, M., Bellare, M., & Rogaway, P. (2018). The security of end-to-end encryption protocols. *Journal of Cryptology*, 31(3), 817-866.
2. Marlinspike, M., & Perrin, T. (2016). The Signal Protocol. Retrieved from <https://signal.org/docs/>
3. Cohn-Gordon, K., Cremers, C., Dowling, B., Garratt, L., & Stebila, D. (2017). A Formal Security Analysis of the Signal Messaging Protocol. *Journal of Cryptology*, 33, 191–246. <https://doi.org/10.1007/s00145-019-09345-0>
4. Bernstein, D. J., et al. (2023). Kyber: Module-Lattice-Based Key Encapsulation Mechanism. *NIST Post-Quantum Cryptography Standardization*. <https://csrc.nist.gov/projects/post-quantum-cryptography/selected-algorithms-2022>
5. Langley, A., Mavrogiannopoulos, N., & Perrin, T. (2018). ChaCha20 and Poly1305 for IETF Protocols. *RFC 8439*. <https://datatracker.ietf.org/doc/html/rfc8439>
6. Goldberg, S., & Van Geest, S. (2018). XChaCha: eXtended-nonce ChaCha and AEAD_XChaCha20_Poly1305. *Internet-Draft*, Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/draft-irtf-cfrg-xchacha>
7. Ermoshina, K., Musiani, F., & Halpin, H. (2017). End-to-End Encrypted Messaging Protocols: An Overview. In *Third International Conference, INSCI 2016, Internet Science* (pp. 244–254). <https://inria.hal.science/hal-01426845v1>
8. Termond, J. (2024). *Security oriented chat application development* [Bachelor's thesis, Mid Sweden University].
9. Cohn-Gordon, K., Cremers, C., Dowling, B., Garratt, L., & Stebila, D. (2017). A Formal Security Analysis of the Signal Messaging Protocol. *Journal of Cryptology*, 33, 191–246. <https://doi.org/10.1007/s00145-019-09345-0>
10. Gidney, C., & Ekerå, M. (2017). Breaking RSA encryption with a quantum computer. *Nature*, 549(7671), 222–226. <https://doi.org/10.1038/nature23461>
11. National Institute of Standards and Technology. (2023). *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism (Kyber) – Draft*. Gaithersburg, MD, USA. Retrieved from <https://csrc.nist.gov/CSRC/media/Publications/fips/203/draft/documents/fips-203-draft.pdf>